

1. List all distinct departments in the students table

Syntax

```
SELECT DISTINCT department  
FROM students;
```

Output

department
HR
IT
Finance

2. Get the average age of students per department

Syntax

```
SELECT department, AVG (age) AS avg-age  
FROM students  
GROUP BY department;
```

Output

department	avg-age
IT	20,5
HR	22
Finance	23

3. Show department with more than 1 student

Syntax

```
SELECT department, COUNT(*) AS student-count  
FROM students  
GROUP BY department  
HAVING COUNT(*) > 1;
```

Output

department	student-count
IT	2
HR	2

4. Get all students whose age is between 21 and 23

Syntax

```
SELECT student-id, name, age, department  
FROM students  
WHERE age BETWEEN 21 AND 23;
```

Output.

student-id	name	age	department
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	24	HR.

23
5. List all students in the IT or HR department who are older than 21.

Syntax

```
SELECT student-id, name, age, department  
FROM students  
WHERE department IN ('IT', 'HR') AND age > 21;
```

Output.

Student-id	name	age	department
2	Bob	22	HR
5	Eve	22	HR.

6. Show total credits per department, only for departments with more than 5 total credits

Syntax

```
SELECT department, SUM (credits) AS total-credits  
FROM Courses  
GROUP BY department  
HAVING SUM (credits) > 5;
```

Output.

department	total_Credits
IT	11

7. List all courses that do not have 4 credits

Syntax

SELECT Course-id, Course-name, department, Credits
FROM Courses
WHERE NOT Credits = 4;

Output.

Course-id	Course-name	department	Credits
101	SQL Basis	IT	3
104	Excel	Finance	2
105	Statistics	HR	3.

8. Show the top 3 courses by credits in descending order

Syntax

```
SELECT Course-id, Course-name, credits  
FROM Courses  
ORDER BY credits DESC  
LIMIT 3;
```

Output.

Course-id	Course-name	Credits
102	Python	4
103	Data Science	4
101	SQL Basics	3

9. Get the maximum, minimum and average grade across all enrollments

Syntax

```
SELECT MAX (grade) AS max-grade,  
       MIN (grade) AS min-grade,  
       AVG (grade) AS avg-grade  
FROM enrollments;
```

Output

max-grade	min-grade	avg-grade
90	78	84.6

10. Count how many enrollments exist per course

Syntax

```
SELECT course-id, COUNT (*) AS enrollment-count  
FROM enrollments  
GROUP BY course-id;
```

Output.

Course-id	enrollment-count
101	1
102	1
103	1
104	1
105	1

1) find total salary and total bonus per department

Syntax

```
SELECT department, SUM(salary) AS total-salary,  
                SUM(bonus) AS total-bonus  
FROM salaries  
GROUP BY department;
```

Output

department	total-salary	total-bonus
IT	122000	10500
HR	109000	7500
Finance	70000	6000

12. Show departments where average salary is above 55 000

Syntax

```
SELECT department, AVG (salary) AS avg-salary
FROM salaries
GROUP BY department
HAVING AVG (salary) > 55000;
```

Output

department	avg-salary
IT	61000
Finance	70 000

13. List employees whose salary plus bonus is greater than 60 000

Syntax

```
SELECT employee_id, name, salary, bonus, salary + bonus  
      AS total_compensation  
FROM   salaries  
WHERE  salary + bonus > 60000;
```

Output.

emp-id	name	salary	bonus	total-compensation
1	Tom	60000	5000	65000
2	Spike	70000	6000	76000
3	Tyke	62000	5500	67500

14. Show total and average budget per department
Only include departments with average budget
above 70 000

Syntax

```
SELECT department SUM (budget) AS total-budget  
                AVG (budget) AS avg-budget  
FROM projects  
GROUP BY department  
HAVING AVG (budget) > 70 000;
```

Output

department	total-budget	avg-budget
IT	27 0000	135 000
Finance	80 000	80 000

15. List all projects with budgets between 50 000 and 120 000, excluding the Marketing department.

Syntax

```
SELECT project-id, project-name, department, budget
FROM projects
WHERE budget BETWEEN 50 000 AND 120 000
AND department <> 'Marketing';
```

Output:

project-id	project-name	department	budget
1	AI App	IT	120 000
2.	Payroll system	Finance	80 000
3.	HR Portal	HR	50 000